Privacy-Preserving Federated Learning for Intrusion Detection in IoT Networks

Eliot Deacon

Student ID: 730008182

Abstract

The internet of things has experienced rapid growth in the past decade and is expected to continue growing. This presents new opportunities for cyber-criminals who can indiscriminately scan the internet for insecure devices and perform a range of attacks. These devices are often used in homes, healthcare and critical infrastructure, making their security of vital importance. Federated Learning has been proposed as a technique for efficient collaborative intrusion detection across many clients. However, it is vulnerable to inversion attacks whereby client updates reveal their local training data. Previous works that design FL-based IDSs naively employ differential privacy to mitigate inversion attacks. This report serves as a foundation by which to study the privacy-utility tradeoff in more depth; I examine how differential privacy affects model accuracy, bias and scalability and measure the level of privacy it achieves using inversion attacks. I demonstrate that varying federated learning settings also has a privacy-preserving effect that should be understood before applying DP mechansims.

☑ **I certify that all material in this dissertation which is not my own has been identified.**

Signature: _____

Contents

1 Introduction

In 2016 several internet infrastructure companies were targeted in a series of high-profile DDoS attacks: the mirai attacks [1]. Attackers had scanned the internet for insecure IoT devices, compromising 600,000+; they used this botnet to launch the DDoS. This event highlighted the need for improved security measures in the IoT. It's not just internet infrastructure that is threatened. IoT devices are used in critical infrastructure [33], agriculture [34], healthcare [30][33], vehicles [32] and smart homes. Compromising these devices can expose personal information, and affect vulnerable populations. In the case of medicine, vehicles and critical infrastructure, attacks could be especially damaging [35][36].

Intrusion Detection Systems are a necessary step in securing these devices and networks. Traditional NIDS's (network intrusion detection systems) need clients to send network traffic data to a central server for analysis, putting a strain on the network and violating privacy. Federated Learning offers an efficient and private alternative.

1.1 Preliminaries

1.1.1 Federated Learning

Federated Learning is a subset of distributed optimisation techniques: a central server coordinates the learning between multiple clients; it initialises a model, sends the model to its clients who then train locally and send the updated models to the server, the server then aggregates the updated model parameters according to some scheme. This is repeated until convergence, or for a set number of rounds. By only exchanging model updates, the strain on the network is greatly reduced and client's network traffic is no longer directly exposed.

FedAvg: McMahan et. al. were the first to propose federated learning, they introduced the FedAvg algorithm: A central server initialises a model w_0 and sends the model to clients who train on their local datasets: $w_{t+1}^k \leftarrow w_t - \eta \nabla \ell(w_t; X^k)$ for some number of epochs and batches. The server receives the model updates and takes a weighted average: $w_{t+1} \leftarrow \Sigma_{k \in C_t} \frac{n_k}{m_t} w_{t+1}^k$ where C_t is the set of clients sampled at timestamp t and n_k/m_t is the size of the clients dataset over the size of the global dataset at that timestamp.

FedProx: A significant problem in federated learning is handling non-i.i.d local datasets. The FedProx aggregation scheme [22] was designed to address and limit the negative effects of heterogeneous data on training. It works by sending the clients the regularisation term proximal-μ, clients then add the term $\frac{\mu}{2}||w - w^t||$ to the loss function to prevent client model updates w from straying too far from the initial global model for that round w^t.

Although client data is not directly sent to the central server in FL, model updates can reveal client data under the right conditions. Deep Leakage from Gradients and similar attacks have been proposed to recover client training data from model updates during distributed training.

1.1.2 Inversion Attacks

DLG: Zhu et. al. [11] showed that exchanging gradients during distributed training can expose private training data. A server, with access to the original model and a client's updated gradients trains the model on dummy data and attempts to match the gradients in its own training with the client's gradients. It is often expressed as an optimisation problem: $\min_{x',y'} ||\nabla W' - \nabla W||_2$ where x', y' are the dummy data and dummy labels, respectively, and $\nabla W' = \partial \ell(F(x', W), y')/\partial W$. The attack relies on a close mapping between the model gradients and the training data. This is effective for smaller batch sizes and smaller local training rounds and simple data, however as data becomes more complex, batch sizes increase, and the computation per client side increases, the landscape drifts and a close mapping is lost.

Improvements: Several extensions to DLG have been proposed including attacks specific to FL schemes. Extensions propose cosine-similarity as a better distance metric between gradients since the direction of updates could be more important than the Euclidean distance [8]. Since federated learning typically uses more local computation than traditional distributed optimisation (the assumed setting for DLG), the close mapping the attack relies on is lost. [12] proposes an attack that uses updates across rounds to improve the learning, rather than performing the attack after one update.

Recovered network traffic data can be used to tailor attacks that avoid detection by using timing patterns from recovered benign data [23].

1.1.3 Differential Privacy:

ϵ-differential Privacy defines an upper bound on how much information a privacy-preserving mechanism reveals [24]:

$$\left| \ln \left(\frac{\mathbb{P}[T_A(x) = t]}{\mathbb{P}[T_A(x') = t]} \right) \right| < \epsilon \tag{1}$$

Where x and x' are adjacent datasets, $\mathbb{P}[T_A(x) = t]$ is the probability the mechanism T produces the transcript t (a record of the observable outputs) and $\mathbb{P}[T_A(x') = t]$ is the probability the mechanism produces the same transcript when ran on x'. Later extensions use a relaxed definition that introduces a δ term, representing the probability of recovering data [25].

The sensitivity of a dataset is defined as the maximum absolute distance between the adjacent datasets: $S = ||f(x) - f(x')||_2$. The Gaussian Mechanism achieves ϵ-differential privacy by injecting random noise into the dataset x; tuned to the sensitivity: $T(x) = f(x) + \mathcal{N}(0, \sigma^2 S^2)$.

In federated learning, noise is typically added to the model parameters after clients have trained on their local data, although it can be applied to the clients local training data instead. I'm not aware of any method that efficiently computes the sensitivity of a dataset; it might not be possible, instead a sensitivity is enforced by clipping the model parameters:

$$\Delta \bar{w}^k = \Delta w^k / \max \left(1, \frac{||\Delta w^k||2}{S} \right)$$

[2] apply this mechanism using a trusted third party.

$$w_{t+1} = w_t + \frac{1}{m_t} \left(\sum_{k=0}^{m_t} \Delta \bar{w}^k + \mathcal{N}(0, \sigma^2 S^2) \right) \tag{2}$$

The third party clips and sums the client updates, and adds noise tuned to the sensitivity. The server then approximates the true average with $1/m_t$, where m_t is the number of clients involved in training at time step t, and updates the model parameters.

Local Differential Privacy: Instead of using a trusted third party, clients clip model parameters and inject noise locally. LDP leads to slower convergence times and decreased overall model accuracy.

1.2 Literature Review

Privacy-Preserving FL: Mechanisms proposing homomorphic encryption (HE) and secret-sharing [7][6] introduce additional computational inefficiency, algorithmic complexity and struggle to scale. [5] use differential privacy and a secret sharing technique – this privacy mechanism also requires a trust authority to coordinate; the authors claim that applying the Gaussian mechanism to gradient updates satisfies the requirements of DP and is thereby theoretically secure. Fed+ is a novel aggregation algorithm which better stabilises differentially private learning [3]. This paper provides an empirical

study of the feasibility of DP-enabled FL in IDSs for the internet of things. FedDef [4] uses inversion attacks and adversarial attacks to quantify privacy.

Of these studies, Fed+ and FedDef are the closest to comprehensive investigations into the privacy-utility trade-off. Fed+ focuses on quantifying the utility of differential privacy but fails to show how DP scales across clients or how it affects the IDSs bias. The authors rely on the definition of DP as a metric for privacy and the Pearson correlation coefficient (pcc) between original and perturbed weights. FedDef focuses on quantifying privacy, improving upon Fed+'s privacy measure by empirically measuring recoverable information using attacks. However, the authors rely on older datasets that aren't representative of the heterogeneity of IoT networks, and the paper is intended to measure the privacy of their own novel privacy mechanism rather than studying DP.

Studies of the Privacy-Utility Tradeoff: Early papers proposing inversion attacks in deep learning and FL relied on idealised conditions; effective only for small batch sizes and few rounds of learning and this was used to argue that FL could still be privacy-preserving on its own. Geiping et. al. [8] demonstrated that image recovery is possible in federated learning beyond highly idealised settings, recovering images in "deep, non-smooth and trained architectures ... and even in batches of 100 images" (though not all images were recovered). The authors argue that inversion attacks are likely to improve and that shortcomings of early attacks should not be used as evidence that FL doesn't require additional privacy-preserving techniques – that DP might still be the only way to guarantee privacy. But DP has been shown to decrease model accuracy [14] and disproportionately affect the accuracy of complex and under-represented classes [9].

Attacks can be somewhat resistant to DP [11], but it has also been shown that inversion attacks can struggle without DP and with more complicated inputs and optimisation techniques [12], Geiping et. al. also measure this effect [8]. It is clear that differential privacy alone does not provide an accurate or useful measure of privacy in deep learning; that varying model architectures, learning techniques and training data also has a privacy-preserving effect (that is not to say it is a privacy-preserving mechanism on its own). These variations will occur naturally across different deep learning tasks. We can, therefore, benefit from domain-specific analyses in the privacy-utility trade-off for FL and from improved measures of privacy in deep learning. The difficulties in deploying DP [13] and the negative effect DP mechanisms have on model accuracy, bias and learning stability are additional incentives to find a better measure of privacy instead of just measuring how perturbed model updates are.

1.3 Motivation

This report aims to provide a starting point that can inform future researchers and security professionals about the privacy-utility trade-off issues inherent in IoT intrusion detection and suggest future directions to improve our understanding of this issue. I intend to measure how architectural decisions, learning techniques and privacy-preserving mechanisms affect the scalability, accuracy and bias of IDSs. I extend the work of FedDef by using a dataset more representative of heterogeneity in IoT networks and I extend the work of Fed+ by measuring learning stability with DP, as well as how it scales across clients and affects the IDSs bias. This is the first study, that I am aware of, to provide a full overview of these issues and how they relate.

1.4 Aims & Objectives

Aim-1: To design and implement a competitive model for IoT network intrusion detection.

- **Obj-1.1:** Achieve an average baseline accuracy $\geq 90\%$ (no privacy mechanism)
- **Obj-1.2:** Converges in ≤ 50 rounds of training
- **Obj-1.3:** Macro-Recall > 0.8 (detecting each attack is equally important regardless of sample size).

Obj-1.4: F1-Score > 0.8

Aim-2: To provide an empirical study of the privacy-utility trade-off; clearly demonstrating the conditions that affect the NIDS performance, stability and scalability.

Obj-2.1: Show how model performance scales across number of clients for different amounts of noise (including no noise).

Obj-2.2: Show how the amount of recoverable information scales across number of clients, batches, and epochs by testing against inversion attacks (DLG, Cosine Similarity). Measure Euclidean distance and PCC.

Aim-3: To design an efficient privacy-preserving framework that accurately classifies network traffic on an IoT network without exposing clients' data.

Obj-3.1: Communication Efficiency: ≤ 100 rounds of training

Obj-3.2: Accuracy remains stable across many clients.

Obj-3.3: $\geq 80\%$ average accuracy

Obj-3.4: Macro-Recall > 0.7

Obj-3.5: Euclidean between original and recovered inputs > 1 and, $-0.1 < PCC < 0.1$.

Stretch-1: To extend the study to use adversarial attacks to find how much recovered information is needed to avoid detection.

Stretch-2: Implement custom attacks for network traffic training data recovery to improve the reliability of privacy metrics.

2 Methodology & Design

2.1 Threat Model

We consider an honest but curious server that coordinates learning among clients: it performs the intended function but attempts to recover private information from the clients. Additionally, we consider any third-party with access to client updates to be actively trying to recover client's data. Each client collects traffic from a network of IoT devices, these clients could be base stations, servers or gateways. Clients are assumed to be heterogeneous; differing in topology, networking protocols and application.

2.2 Chosen Dataset - ToN-IoT

ToN_IoT [17] is a collection of datasets obtained from four data sources: IoT and IIoT sensor data, OS logs, packet capture files and bro logs. For the purposes of building a NIDS we are only concerned with network traffic datasets. The network traffic datasets used in this project are represented as flows; each flow is a summary of a conversation between network nodes: flows are generally preferred to packet capture in data analysis since they require less storage and can be kept for much longer periods of time [16]. This makes the dataset more representative of clients local datasets.

Justification: Previous work in designing FL-based IoT NIDSs relied on older, bespoke network traffic datasets that did not suitably represent the heterogeneity of IoT network [4][7][15]. As such results from these papers aren't as representative of how an FL-based NIDS would behave in practice. ToN_IoT was designed specifically to address this, using a diverse set of IoT and IIoT devices,

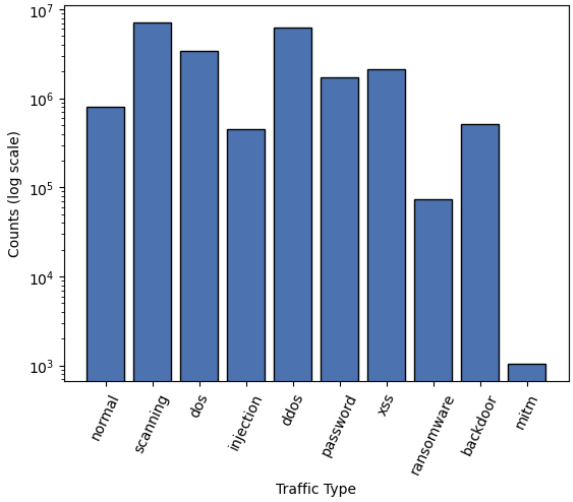


Figure 1: Class Counts

virtualisation technologies, operating systems, attacks and a testbed that simulates IIoT and industry 4.0 networks. This makes it ideal for simulating clients with non-i.i.d. local datasets in a federated learning setting.

Limitations: Malicious network traffic has been simulated in sequence (normal network traffic runs throughout the simulation). This makes it harder to partition the dataset among clients while maintaining temporal patterns and is not representative of realistic network traffic. There is also a significant class imbalance which could lead towards attacks with larger footprints.

2.3 Pre-Processing Pipeline & Partitioning

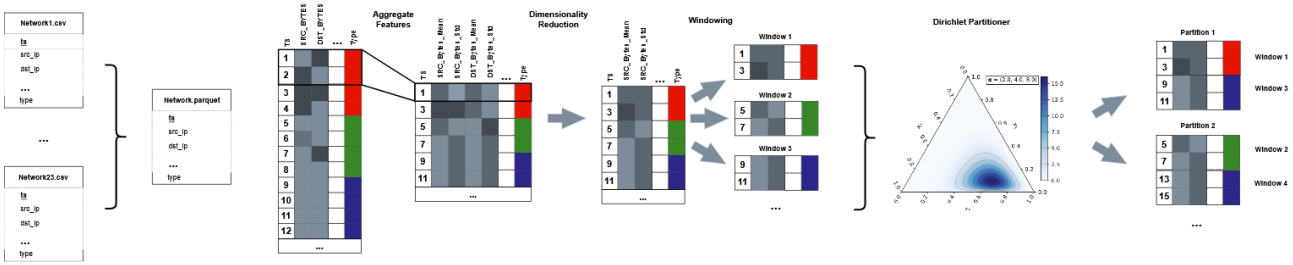


Figure 2: Pre-Processing Pipeline[1]

The ToN_IoT network traffic dataset contains roughly 22 million records (flows). Since each flow represents a conversation on the network, attacks that participate in more conversations, like scanning, dominate the class distribution, see Figure 1. This pipeline aims to reduce the size of the dataset while maintaining important information, and to partition the dataset among clients so that each client has a non-i.i.d. local dataset (to simulate realistic FL conditions) while maintaining realistic temporal patterns.

Aggregation & Dimensionality Reduction: This step divides the dataset into windows of equal size and collapses the window into a single record containing a statistical summary of its features. I use a step size equal to the window size since the dataset needs to be greatly reduced. Mitm records are disparate so they are unlikely to be the final record in the window; to mitigate this my labelling rule takes the final record in a window unless a mitm record is in the window, then the aggregated type

[1]Image Credit for the Dirichlet Distribution Ternary Plot: Nerebur – Wikipedia Commons

is "mitm". After aggregating features into statistical summaries, the feature space will be increased. To avoid over-fitting I remove highly correlated features using a Pearson correlation matrix.

Windowing: Instead of partitioning on individual records, I divide the dataset into windows, label according to same strategy used in aggregation, and partition on the windows. This maintains temporal patterns within the windows for local datasets. Since network traffic is event based, windows can contain temporally unrelated data. I handle this by introducing a Δt parameter measuring the difference in timestamps between each record; if a window contains a Δt beyond a set threshold the window is removed. The threshold is set low enough s.t. windows with impulsive attack types aren't removed but high enough that the number of windows to remove is small.

Dirichlet Partitioner: The Dirichlet distribution is used to partition the global dataset of windows into non-i.i.d. local datasets. Intuitively, it describes the probability distribution over proportions of classes in a client's dataset. The parameter $\alpha = (\alpha_1, \ldots, \alpha_k)$ determines the probability distribution (k is the number of partitions); a high α_c means client c is more likely to have a higher proportion of a class in their dataset. Clients sample the Dirichlet distribution, forming a matrix $[p_{ij}]_{10\times K}$, for each of the ten traffic types, where p_{ij} represents client j's proportion of class i:

$$\begin{bmatrix} p_{11} & p_{12} & \cdots & p_{1,K} \\ p_{21} & p_{22} & \cdots & p_{2,K} \\ \vdots & \vdots & \ddots & \vdots \\ p_{10,1} & p_{10,2} & \cdots & p_{10,K} \end{bmatrix}$$

The matrix describes exactly how the global dataset is split; we build a dictionary (k, I) where k is the partition id and I is a list of indices of windows from the global dataset. Each I represents a local dataset with the class distribution described by that clients column in the matrix. When the experiment runs, clients get a partition id from the server, they get window indices from the partitioner, and get their windows from the global dataset.

The result of the pipeline is that each client has windows sampled at random from the Dirichlet distribution where each window contains a contiguous segment of network traffic data from the original dataset.

Justification: Aggregation helps to reduce dimensionality while maintaining crucial information. Categorical features that could usually cause over-fitting, like IP-addresses, can be aggregated into metrics representing the distribution of IP-addresses in a window. This maintains information that is crucial for many attacks. This step somewhat reduces bias as well, although not being a direct fix: by converting the domain from flows to time the bias becomes attacks that take longer (i.e. DDoS) rather than attacks that have more conversations on the network (i.e. Scanning). This is preferable since the variations in the duration of attacks are less than the variations in the amount of flows an attack takes up , see Figure 6 in Section 3.2.

Limitations: The partitioning approach does not maintain temporal patterns across windows so clients datasets aren't fully realistic time-series'. ToN_IoT itself is not a fully realistic time series since traffic types were collected sequentially and with some large time jumps where the authors weren't simulating any traffic on the testbed.

Success Criteria: If Federated Learning on the processed and partitioned dataset is accurate and generalises well – despite the class imbalance in the original dataset – s.t the first aim of the project is achieved, then the pipeline is considered successful in generating local datasets with realistic temporal data and mitigating bias.

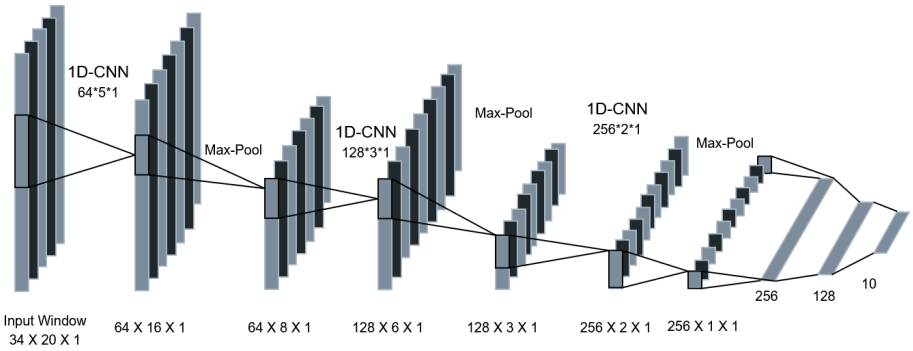


Figure 3: Model Architecture

2.4 Model Design

The model is composed of three convolutional layers with batch normalization and max pooling, and an MLP on top for classification. Convolutions and pooling layers identify temporal patterns of increasing complexity; batch normalization is added to reduce internal covariate shift; ReLu functions introduce non-linearity. The output of the convolutional layers is flattened into a 1D-vector using dynamic pooling and inputted into the MLP classifier. I use dynamic pooling to be able to vary the window size during tuning and experiments without altering the architecture. A cross-entropy function is used for multi-class classification.

Model Changes for DP: When running differential privacy experiments, the batch normalization layers are removed. This is because they add excessive regularisation when combined with the clipping from the privacy mechanism.

Justification: 1D-CNNs have been shown to be highly accurate and efficient at time series classification tasks, anomaly detection [26], and FL-based network intrusion detection [7][27]. The architecture complements the processing pipeline; it is designed to work effectively with windows of data and if clients all had realistic time series data as local datasets, the model would still work.

Limitations: CNNs aren't effective for long-term attacks; this would require models with longer memories like transformers, since the pipeline generated temporally unrelated windows this is not an issue. I use a supervised training method and I only train on the attacks collected in the ToN_IoT dataset so the IDS would struggle to detect zero-day attacks compared to anomaly-detection approaches – this is an active research area and not the focus of this project.

2.5 Federated Learning Framework Design

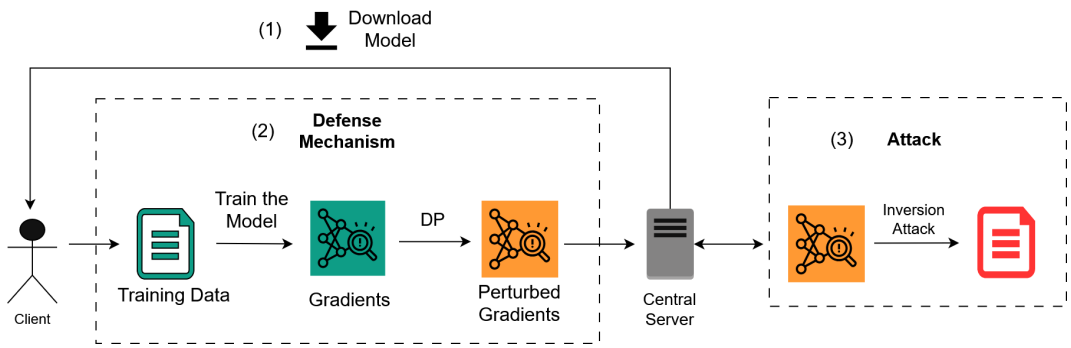


Figure 4: Framework Architecture

The central server initialises the model and distributes it to whichever clients are participating in that

round of learning. The clients train the model on their local datasets, then they clip and inject noise – tuned to the sensitivity – to the parameters. The server then aggregates the clients' parameters and attacks the updates individually (the order here doesn't matter).

I use the Gaussian mechanism (Section 1.1.3) to add noise to the model parameters and I experiment with both FedAvg and FedProx (to reduce bias) as aggregation scheme. I implement DLG, deep leakage with cosine similarity, and an extension of inversion attacks to federated learning to gain measures of privacy. The extended attack takes place over all rounds of learning rather than being isolated to a single round of updates at a time.

Federated Optimisation & Learning Techniques: I use SGD as a baseline throughout the experiments, and Adam to understand how improved optimisation techniques effect DP and inversion attacks. Adam is less sensitive to initialisation [28] than SGD and could therefore mitigate the effect DP has on bias. The attacks (section 1.1.2) use an LBFGS optimiser [19]; the same optimiser used in the original DLG attack, the algorithm was designed to optimise a function w.r.t a vector x – similar to the objective function of DLG, and has been shown to be effective at parameter estimation [18] making it ideal for inversion attacks.

2.6 Experimental Methodology

Experiments 1 and 2: Tuning FL Parameters (Adam & SGD)

$lr \in \{0.1, 0.03, 0.01, 0.001, 0.0001\}$
$LocalEpochs \in \{1, 3, 5, 10\}$
$BatchSize \in \{16, 32, 64\}$
$WindowSize \in \{20, 30, 40, 50\}$

Perform a random search over the parameters for Adam and SGD optimizers; each 30 runs. There are 240 combinations so each random search covers $\approx 16.7\%$ of the search space. This balances computational limitations and coverage. Measure the final accuracy, loss and f1-score of the model after training .

Experiment 3: Tuning Differential Privacy

$ClippingNorm \in \{0.5, 1, 2, 5, 10\}$
$Sensitivity \in \{0.5, 1, 2, 4\}$
$Delta \in \{1 \times 10^{-5}, 1 \times 10^{-6}\}$

Perform a random search of 20 runs over the parameters. There are 40 runs so the search gives a 50% coverage of the parameter space. Measure the final accuracy and f1-score of the model. For each run I train on 150 clients; the most clients I can simulate before partitions become too sparse. DP is unstable for fewer clients since local datasets diverge more with noise – using the maximum number of clients provides a baseline for DP.

Experiment 4: Tuning Attacks

$Rounds \in \{10, 25, 50\}$
$MaxIters \in \{5, 10, 20\}$
$HistorySize \in \{10, 50, 100\}$

Perform a random search of 20 runs over the parameters from baseline conditions: $BatchSize = 1$, $LocalEpochs = 1$, $LocalBatches = 1$ and SGD for the local optimizer. This gives $\approx 74\%$ coverage of the parameter space

Experiment 5: Varying Noise across Clients

$\epsilon \in \{10, 7.5, 5, 2.5\}$

$NumClients \in \{5, 10, 25, 50, 75, 100, 150\}$

$\forall \epsilon, NumClients$ train the model using the tuned parameters for Adam and SGD. Measure accuracy and loss over rounds and final accuracies over $NumClients$.

Experiment 6: Varying Noise across Prox-μ

$\epsilon \in \{10, 7.5, 5, 2.5\}$
$Prox\mu \in \{0, 0.001, 0.0001, 0.00001\}$

$\forall Prox\mu$, ϵ train the model over 150 clients, using the tuned parameters from Experiments 1, 2 and 3. Measure the accuracy-per class to understand how FedProx can mitigate bias. Use baselines of 10 clients without noise and 150 clients without noise. Minority attack classes will become disparate when partitioned among many clients - the baseline of 10 clients and no noise is used to measure the per-class accuracy in more realistic local datasets. Since DP is unlikely to be effective for 10 clients I also use a baseline of no noise and 150 clients.

Experiment 7: Measure Recoverable Information for Different Local Training Parameters

$LocalEpochs \in \{3, 5, 7, 10\}$
$LocalBatches \in \{8, 16, 32, 64\}$
$BatchSizes \in \{8, 16, 32, 64\}$
$WindowSize \in \{30, 40, 50\}$

Starting with a baseline of $LocalEpochs = 1$, $LocalBatches = 1$, $BatchSize = 1$, $WindowSize = 20$ I vary each parameter individually to understand its affect on privacy. I run this experiment for 10 clients and measure the average and minimum MSE, and the PCC between client inputs and recovered inputs.

Experiment 8: Measure Recoverable Information for Different Values of ϵ

$\epsilon \in \{10, 7.5, 5, 2.5\}$

Starting with a baseline of $LocalEpochs = 1$, $LocalBatches = 1$, $BatchSize = 1$, $WindowSize = 20$ I vary ϵ, measuring the minimum and average MSE and the average PCC between clients inputs and recovered inputs. The experiment is run for 150 clients to stabilize differentially-private learning and attacks are run on a sample of 10 clients for computational efficiency.

Steps to ensure reproducibility:

- Extensive logs are taken for each each experiment containing a full list of parameters and settings for FL, DP and inversion attacks, as well as seeds and experiment names. Each log also contains metrics for accuracy, loss, f1-score, accuracy-per-class and input recovery statistics throughout and across the training rounds. Each experiment script contains a description at the top and additional instructions for reproducible experiments.

3 Development & Experiment

3.1 Flower Framework

Flower [29] is a framework for simulating and deploying federated learning architectures. It provides a wide range of functionality and is highly customizable which is ideal for this project, allowing me to have tighter control at each point in the FL process.

Overview of the Flower Architecture: SuperLink, SuperNode and SuperExec are all long-running processes for managing communication on the network (simulated in our case). For this project,

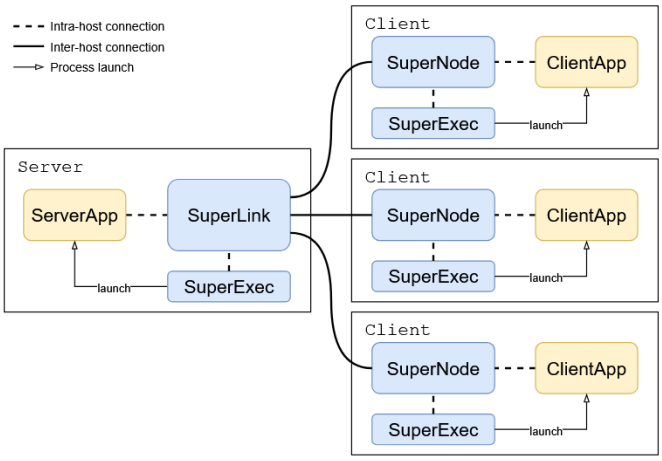


Figure 5: Flower Architecture, Image Credit: Flower Labs

we are only concerned with the ServerApp and ClientApp processes since they define task-specific functionality. If needed, more information on the other processes is available in the flower docs [20].

task.py: This defines the model class (in our case with pytorch), functions for training and testing the model, and functions for loading partitions.

server_app.py: Instantiates the ServerApp class and defines the main server functionality (getting the config, initializing the model, running the strategy) using a decorator pattern.

client_app.py: Instantiates the ClientApp class and defines the training and evaluation functions using a decorator pattern.

pyproject.toml: This is the federated learning config file. By default, it contains the number of local epochs, batch sizes and learning rate, and it can be modified to include other parameters. Flower no longer uses this file to control the number of supernodes in an experiment, instead this is defined in the config.toml file in the .flwr directory.

Flower Datasets: This is a separate library for common dataset management and partitioning tasks in federated learning. Notably, it is integrated with the HuggingFace datasets library and defines a number of partitioners for federated learning experiments.

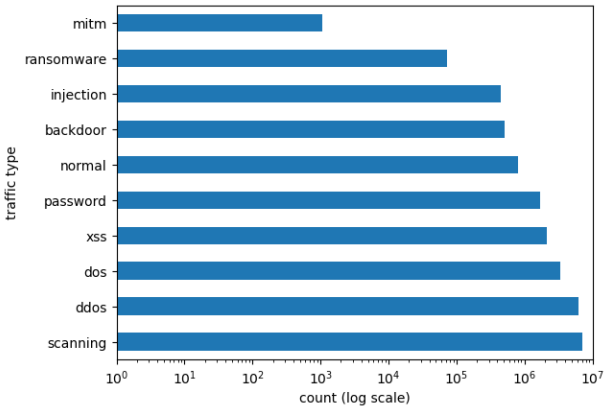
Flower Strategies: The ServerApp itself does not define the federated learning strategy. This is done by Strategy classes inherited from the abstract class Strategy. These classes can be modified for additional functionality by defining a child class and overriding and adding additional methods.

3.2 Pre-Processing, Partitioning and Loading Data

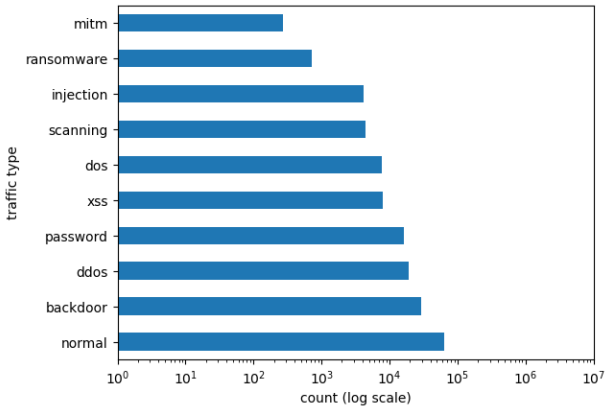
3.2.1 Pre-Processing

The ToN_IoT network traffic dataset is stored as 23 csvs each around 1million records, to process efficiently we first convert the datasets into a single parquet file, this also compresses the dataset significantly. Per feature aggregation over windows is achieved using the tsflex library [21]. TsFlex allows for simple and efficient aggregation over multi-variate time series': it intuitively handles unevenly sampled data (useful for event-based network traffic) and categorical features. After processing, the dataset size is greatly reduced (Figure 6), and the class imbalance is somewhat reduced.

Processing Categorical Features: Most categorical features in the dataset are dead before processing, these are not aggregated. I keep *src_ip, src_port, dst_ip, dst_port* and store the number of unique values in a window, the frequency of the most common value, and its proportion within that window. These statistics allow us to retain useful information without over-fitting.



(a) Class Distribution - Original Data



(b) Class Distribution - Aggregated

Figure 6: Class Distributions

Processing Continuous Features: Many continuous features are also dead. I keep *duration, src_bytes, dst_bytes, src_pkts, dst_pkts* and store the minimum, mean, standard deviation, kurtosis and skewness of the values within a window. Many traffic types are characterized by mostly small values but heavy tails across these features; by storing these measurements instead of direct values I retain important information on the features without inputting extreme values into the model. The minimum of these features within the window is almost entirely 0 except for *src_pkts* so the *min_dst_pkts, min_dst_bytes* and *min_src_bytes* are removed after aggregation.

Aggregating Labels: To aggregate the labels of a window I take the type of the last record unless the window contains one of the minority classes: "mitm" or "ransomware" in which case we take that type (if it contains both the labelling rule uses "mitm") [**ERRATUM**[2]]. We filter features by constructing a pearson correlation matrix and removing highly correlated features.

3.2.2 Loading Data:

Clients call the *load_data()* method (see Figure 11 in the Appendix) to load their local datasets (partitions): The first call to *load_data()* instantiates the partitioner and stores it in a global variable *partitioner*, the global dataset is loaded into a global variable *fds* using the hugging face *load_dataset()* method (for compatibility with flower datasets) and it fits transforms to the global data: standardizing features ($\mathcal{N}(0,1)$) and label encoding traffic types. Since each *ClientApp* is a separate process every client loads the global dataset, fits transforms and instantiates partitioners individually. To ensure the partitions remain consistent across clients, partitioners use seeded RNGs and each client has a unique *pid* which they use to load their partition.

Every subsequent call from a client (and the first call) loads a local dataset from the partitioner using the clients' partition id. This partition is split into local training and testing datasets (a stratified split if possible). These datasets are scaled and labelled by the fitted transforms. Finally, two DataLoader classes (trainloader and testloader) are instantiated using CustomDataset objects and returned to the client.

[2]When making Figure 6 I identified that the ransomware counts were few after aggregation; since ransomware flows appeared within normal traffic and the stepsize is large I assume the final record for most windows during the ransomware attack was "normal". This lead me to include the special case for ransomware during aggregation. The only experiment that would've been majorly affected by this change is Experiment 6: Measuring bias across noise (this was re-run). The other experiments will remain qualitatively no different though F1-metrics should be interpreted with this caveat.

3.2.3 Partitioning

Each *TemporalPartitioner* object has an inner *DirichletPartitioner* object. I override the dataset setter method in the parent class (*Partitioner*) to window the data and set *inner.dataset*; the inner dataset consists of single records for each window in the global dataset. When a *ClientApp* loads a partition for the first time using *TemporalPartitioner*, the inner *DirichletPartitioner* object generates window indices for each partition using the method described in Section 2.3. I use $\alpha = 1$ for all clients, so no client is artificially favoured and heterogeneity is a result sampling proportions from the Dirichlet distribution. The inner partitioner returns indices for the windows to the *TemporalPartitioner* which gets the gets the indices for all records in the windows and returns the local dataset to the client.

3.2.4 Train-Test-Split, Custom Dataset and DataLoaders

After receiving the partition, the client generates a train-test split, stratifying by attack type, if possible. Since experiments are run for many clients local datasets can become sparse and stratified partitioning logic fails, in which case partitions are generated randomly. A *CustomDataset* class defines data loading logic: *CustomDataset.__len__()* returns the windows count and *CustomDataset.__getitem__(index)* returns the windows themselves. A *CustomDataset* is instantiated for the train and test partitions: *train_ds* and *test_ds*, and the *load_data(...)* method returns *DataLoader(train_ds,...)*, *DataLoader(test_ds,...)*.

3.3 FL Framework Implementation

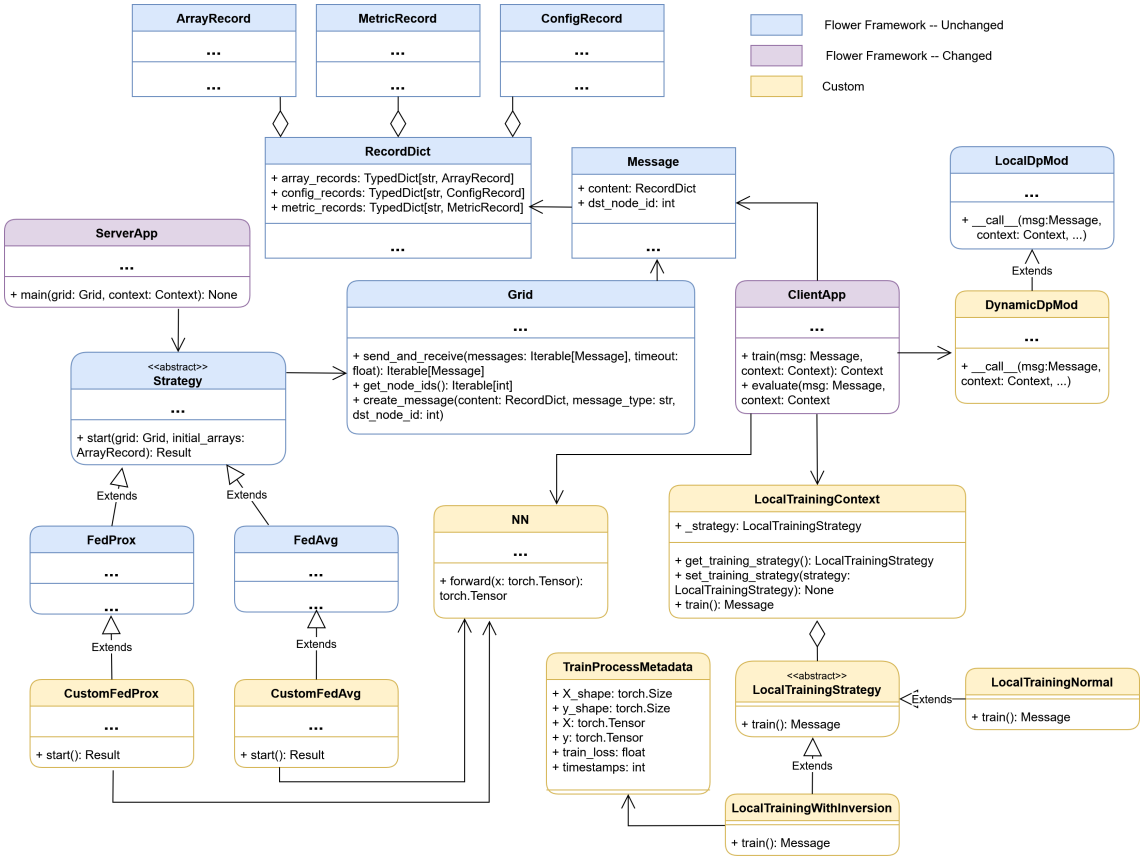


Figure 7: FL Framework: UML Diagram Overview

3.3.1 ServerApp:

The *ServerApp* class is not defined in my implementation; it's a part of the flower library. Instead the standard flower project instantiates a *ServerApp* object, defines the *main()* function and uses a decorator pattern to "upload" it to the *ServerApp* object. This function parses the *pyproject.toml* file then initializes the global model, instantiates and starts a federated learning strategy, and finally saves the trained model. In my implementation I also define some global evaluation methods – these are not used for training.

3.3.2 Strategies:

Strategy classes define the core logic of the federated learning process. Popular strategies are included in the flower framework (FedAvg and FedProx included) but, in our project, they need to be overridden to include inversion attacks. The *start()* method contains the main learning loop. We override this method in the classes *CustomFedAvg* and *CustomFedProx*, see Figure 12 in the Appendix.

Grid: Throughout this loop a *Grid* object is used to send and receive messages. It constructs Messages and interfaces with *SuperLink* to push and pull them to and from the clients.

Message: Each *Message* class is instantiated with a *RecordDict* object and a destination node id as well as some metadata. Message objects are how model parameters, training metrics and training configurations are passed between client and server and vice versa. A *RecordDict* can contain array records, config records and metric records.

ArrayRecord: A typed dictionary *dict[str, Array]* used in flower to exchange model parameters. It is similar to pytorch's *state_dict* and exposes methods to convert to and from numpy arrays and pytorch *state_dict*s.

ConfigRecord: This is usually used to pass federated learning configurations to the clients (e.g. learning rate) in a round of training. The strict definition is that it defines a dictionary of basic types or strings of basic types. In my implementation I also use it to transmit custom training metrics.

MetricRecord: Typically used to transmit results from local training or evaluation back to the server.

Attacks: Attacks, attack evaluation and gradient recovery functions are defined in *attack.py* and imported into the strategy class. Each attack instantiates a *torch.optim.LBFGS([dummy_X,dummy_y]* optimizer. Since LBFGS optimizes over real valued vectors I convert the labels to soft labels; probabilities for each traffic type. Since the model still outputs raw labels I need to use a custom criterion function *soft_label_cross_entropy()* that converts raw logits to probabilities using the log softmax function and then calculates cross-entropy loss. To conduct the multi-round attack, deep copies of the initial model parameters and client gradients are stored per round.

The *FedProx* strategy requires clients to include the term proximal-μ during learning, each training method takes a parameter *prox_mu* whose default value is 0 (FedAvg is the default aggregation scheme).

3.3.3 ClientApp:

I use the strategy design pattern to allow for clean extension to different local training strategies. Similar to *ServerApp*, the *ClientApp* class is not defined in my implementation; it is a part of the flower framework. Instead we define the *train()* function and "upload" it to the *ClientApp* object using the decorator pattern. In the training function we load the model with the parameters received from the server, we load the clients' local dataset using the *load_data()* function (defined in *task.py* and explained above) and check the configuration to see which local training strategy we should use. We then instantiate the *LocalTrainingContext* class passing a *LocalTrainingStrategy* object as the parameter. Two child classes override the *train()* function: *LocalTrainingNormal* and *LocalTrainingWithInversion*.

Each class inherited from *LocalTrainingStrategy* essentially wraps a training method defined in *task.py*. These training methods contain the actual logic for training the model. I have defined two such functions: *train()* and *inversion_train()*. The *inversion_train()* function is written to allow for more control over the training data which is useful in inversion attack experiments, but both functions train in the same way. The *LocalTrainingStrategy* classes store the trained model parameters in an *ArrayRecord* and training metrics in a *MetricRecord* and uses them instantiate a *Message* that is returned to the server.

3.4 Experiment Implementation

Each experiment is implemented in a script that follows the same format: iterate over different parameter settings for the experiment and write the configurations to *pyproject.toml*. Then, for each configuration, execute *flwr run* in the command line. During global evaluation in *server_app.py*, the parameters of the run and the results are logged to a json lines file. Separate scripts interpret the results: iterating over each line in the jsonl file (each representing a run) and recording relevant parameters and results.

4 Results & Discussion

4.1 Model/Framework Tuning and Baseline Performance

Table 1: Hyperparameter Search Results with Adam optimiser

Hyperparameters				**Performance**	
LR	Epochs	Batch Size	Window Size	Acc.	F1
0.001	10	32	20	**0.953**	**0.873**
0.001	3	64	30	0.950	0.764
0.001	10	16	40	0.950	0.752
⋮	⋮	⋮	⋮	⋮	⋮
0.1	10	16	50	**0.410**	**0.058**
0.1	3	64	40	0.407	0.138
0.1	3	64	30	0.394	0.202

Table 2: Hyperparameter Search Results with SGD optimiser

Hyperparameters				**Performance**	
LR	Epochs	Batch Size	Window Size	Acc.	F1
0.01	10	64	20	**0.963**	**0.763**
0.01	3	16	50	0.961	0.755
0.1	5	32	30	0.959	0.841
⋮	⋮	⋮	⋮	⋮	⋮
0.0001	1	32	20	**0.421**	**0.071**
0.001	1	64	40	0.419	0.061
0.0001	3	32	40	0.401	0.059

Table 3: Differential Privacy Parameter Search Results with Adam optimiser

Hyperparameters			**Performance**		
Sens.	Clip	δ	Acc.	F1	Loss
0.5	2	1e-05	**0.910**	**0.716**	**1.295**
0.5	1	1e-05	0.866	0.610	1.368
1	2	1e-06	0.850	0.587	16.10
⋮	⋮	⋮	⋮	⋮	
4	10	1e-05	**0.588**	**0.316**	**654.0**
2	0.5	1e-06	0.588	0.341	4295.3
4	1	1e-05	0.466	0.229	94696.0

Table 4: Attack Hyperparameter Search Results with LBFGS optimizer

Hyperparameters			**Performance**		
Rnds.	Iters	Hist.	Avg. MSE	Min. MSE	Avg. PCC
50	20	10	**1.182**	**1.126**	**0.006**
25	10	50	1.263	1.114	-0.008
25	5	100	1.269	1.139	-0.017
⋮	⋮	⋮	⋮	⋮	
10	20	50	**4.294**	**1.159**	**-0.001**
50	5	10	6.331	1.131	0.001
50	20	10	7.349	1.096	0.003

4.1.1 FedAvg Tuning:

Model tuning was run on 35 FL rounds and 10 clients. The results of tuning (Table 1 and Table 2) show the learning rate to be the most important parameter for both Adam and SGD. Adam achieved the best results with a lower learning rate and SGD with a higher learning rate. The full tuning results are in the appendix (Table 8 and Table 9) they show that Adam performed better with a higher number of local epochs, and that batch and window sizes didn't have a noticeable affect on model performance for both SGD and Adam. Both achieved an accuracy $> 95\%$ on the best run, and an f1-score $> 80\%$.

4.1.2 Differential Privacy Tuning:

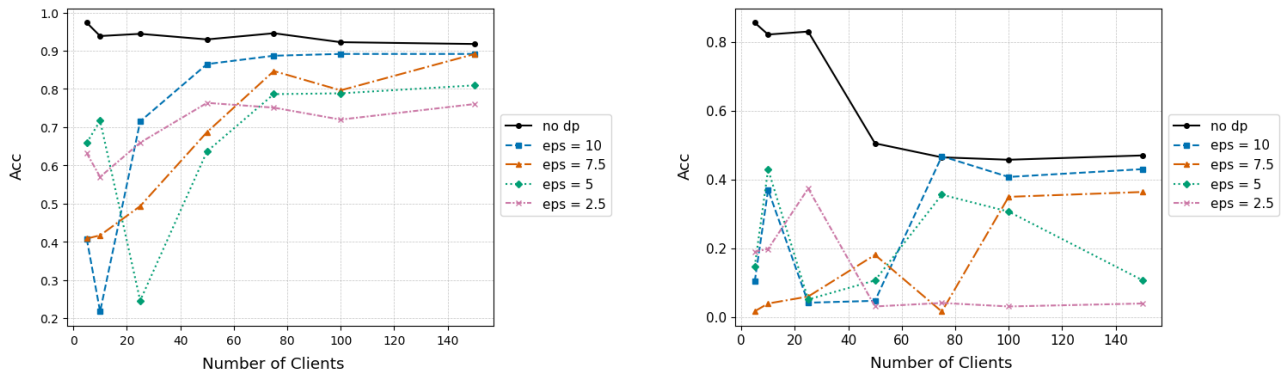
This experiment was run on 150 clients with $\epsilon = 10$ and an Adam optimizer; these are ideal conditions for differential privacy, using very little noise and minimising the heterogeneity of local datasets by using a large number of clients and an optimizer less sensitive to starting conditions. The best results are achieved when the clipping norm is greater than the sensitivity and the sensitivity is small. The δ parameter doesn't have an effect on model performance, although the search space was small. The best tuning achieved a global accuracy of $\approx 91\%$, competitive with non-DP results. However, the final model losses demonstrate highly unstable learning with poor tuning. The disparity between strong accuracy and exploding losses is likely because DP amplifies bias and so the model is more confidently wrong (this is shown in Section 4.2.1).

4.1.3 Attack Tuning:

Attack tuning was performed with *LocalBatches=1, LocalEpochs=1, BatchSize=1*, an SGD optimizer and FedAvg aggregation scheme. The results shown in Figure 4 (full results in the Appendix – Figure 11) demonstrate that the attack is ineffective even in baseline conditions. The results remain mostly the same across different parameters with average and minimum MSE's > 1 and $-0.1 <$ PCC < 0.1, demonstrating no correlation between original and recovered inputs. It seems the attack is not learning beyond the initial random dummy data. Inversion attacks have been shown to struggle with complicated inputs and are generally applied to image recovery tasks on baseline datasets [8][12], while using CNN architectures that can reflect the inputs simplicity in their gradient updates. It is likely that the attack struggles with the complexity of heterogeneous network traffic data and the 1D-CNN architecture used for classification.

4.2 Experiments

4.2.1 How different levels of noise affect learning:



(a) Adam (b) SGD

Figure 8: Accuracy Over Clients

The sparsity of datasets when partitioning among many clients explains the drop off in baseline accuracy; this is more exaggerated while using SGD. Figure 8 shows the accuracy drop from roughly 80% to roughly 50% and remains there. It's unlikely clients will have such sparse datasets in deployment.

For a smaller number of clients differential privacy is unsuitable. The heterogeneity between local datasets is more pronounced with fewer clients and so applying differential privacy causes the model updates to diverge more. This means FL-based IDSs should consider other privacy mechanisms that work well with fewer clients (Homomorphic Encryption, Secure Multi-Party Computation) but may not scale well.

Figure 8 shows that Adam mitigates the effect of bias on accuracy for small values of epsilon whereas the final global accuracy for small values of epsilon (≤ 5) drop toward zero when using SGD. And figure 9 shows that Adam more effectively stabilises learning across ϵ. The missing values (figures (e) and (g)) are where the loss was too high to compute. Although using DP with Adam results in high global accuracies across ϵ, the losses still explode. Again, this is likely because models are more confident in incorrect guesses.

Accuracy per attack class / Bias metrics:
Figure 10 shows the baseline model trained on 10 clients accurately classifies all of the traffic types except for "mitm", achieving a macro-recall of 0.8728. At 150 clients, ransomware attacks are no longer detected. There are few ransomware windows in the global dataset so partitioning these windows among many clients makes ransomware attacks incredibly sparse in local datasets; this is true of mitm windows for small numbers of clients as well. This is a limitation of the pre-processing pipeline and available data, it is not representative of realistic conditions. At 150 clients I achieve a macro recall of 0.7382, this remains high (≈ 0.7) for $\epsilon = 10$ and then drops below 0.7 for more noise (the threshold for *Obj-3.4* in Section 1.4).

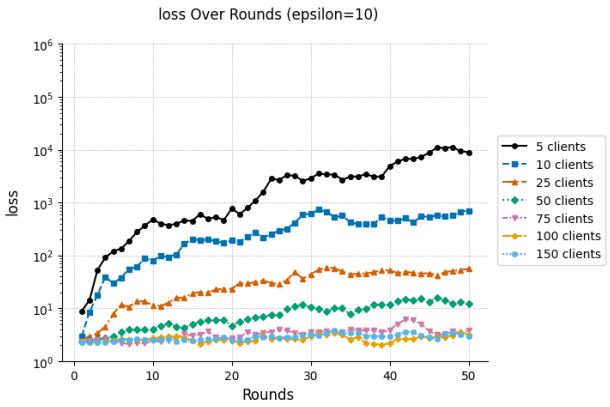
Figure 10 shows that prox-μ successfully reduces bias across all levels of noise although, the bias for small values of ϵ remains high. We can also see that as more noise is injected the accuracy is disproportionately reduced for more complicated attack types with smaller footprints. This mostly aligns with class distributions in the aggregated dataset (Figure 6) with the exception of scanning attacks which is likely more frequent when applying the labelling strategy to windows. This is a significant issue with differentially private FL-IDSs since complex and stealthy attacks often pose higher threats. This is especially true in the case of ransomware.

4.2.2 How much recoverable information is there for different levels of noise:

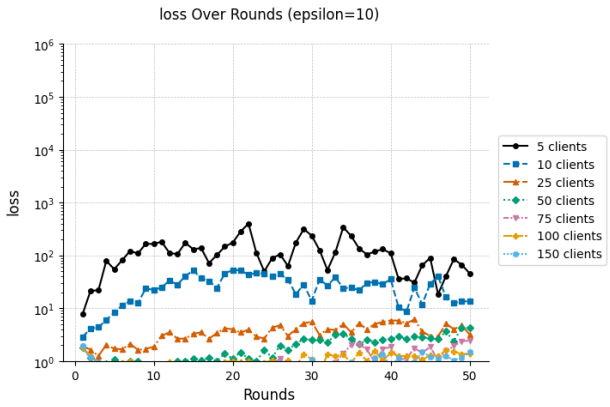
Table 5 shows the average results of two runs of experiment 7. This was run over 10 clients, each using a SGD optimizer; each client was attacked and an average is taken. There were only two FL rounds of training; this is due to computational limitations and because the model doesn't need to be fully trained to measure the attacks' effectiveness. Running inversion attacks, especially on larger input sizes, is computationally expensive hence the small sample size. As such there are some noticeable outliers (862 average MSE for cosine similarity attack on window sizes of 40).

Varying the number of local batches seemed to have the greatest affect on attack performance, with 64 local batches having an average MSE of ≈ 13 compared to the baseline of ≈ 1.5 for the DLG attack, and a slight but noticeable increase for cosine similarity (1.5-2) and multi-round attacks (1.5-5.7). Varying the batch size caused the average MSE for each attack to rise to ≈ 3 and stay there. Varying the window size had little effect on attack performance.

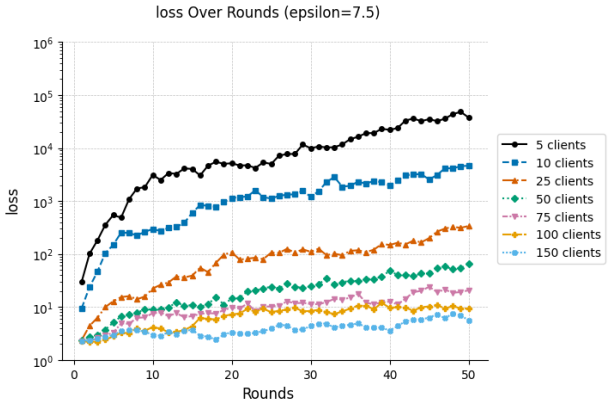
In baseline conditions and across variations in local epochs, the attack doesn't appear to be learning beyond initial dummy inputs. This is evidenced by the similarity in DLG and multi-round attack metrics (each within 3 s.f. of each other). Since the same seeded RNG's are used for all attacks they will start with the same dummy inputs, if attacks don't learn beyond the initialisation, the resulting distances between dummy inputs and actual inputs will be roughly the same. So variations in local



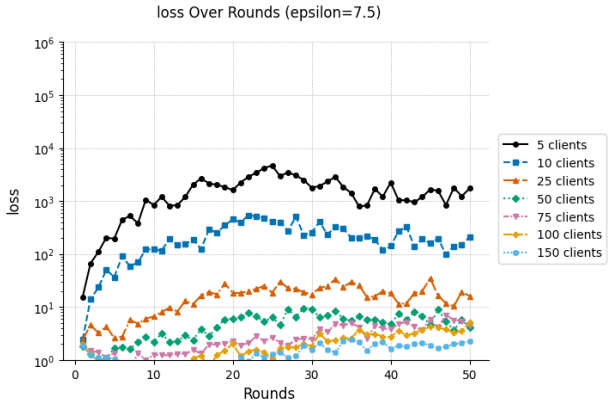
(a) ϵ=10, SGD



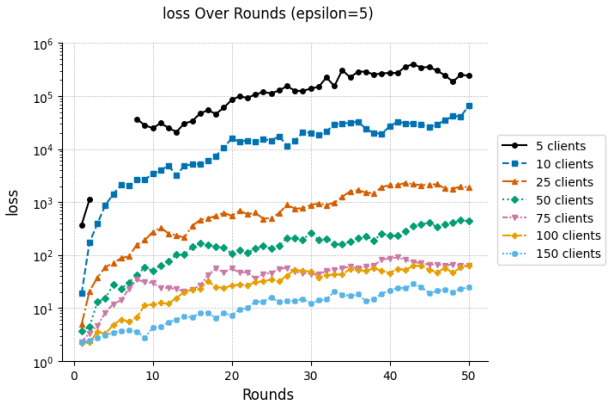
(b) ϵ=10, Adam



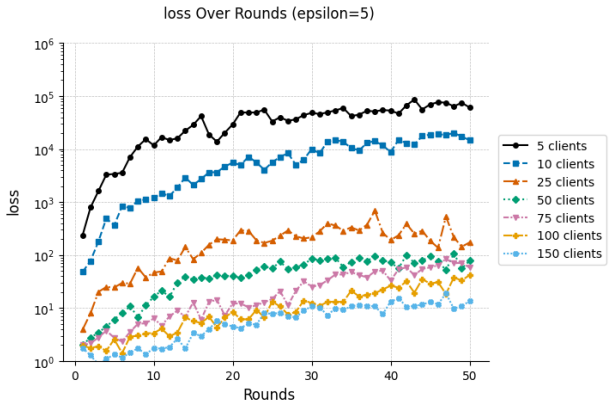
(c) ϵ=7.5, SGD



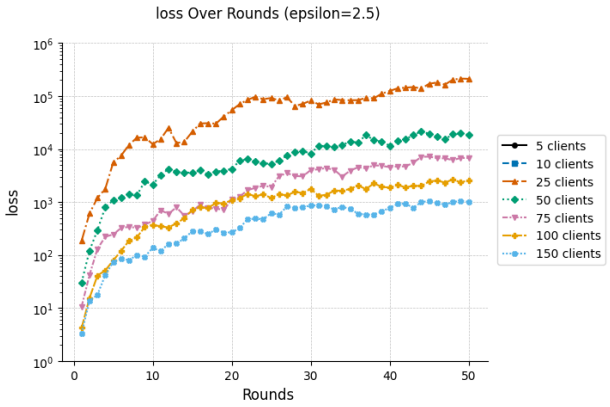
(d) ϵ=7.5, Adam



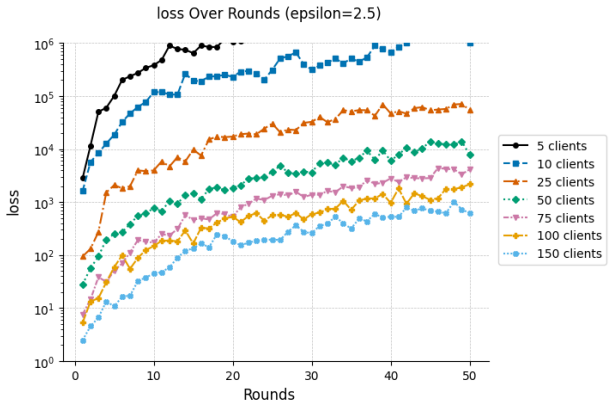
(e) ϵ=5, SGD



(f) ϵ=5, Adam

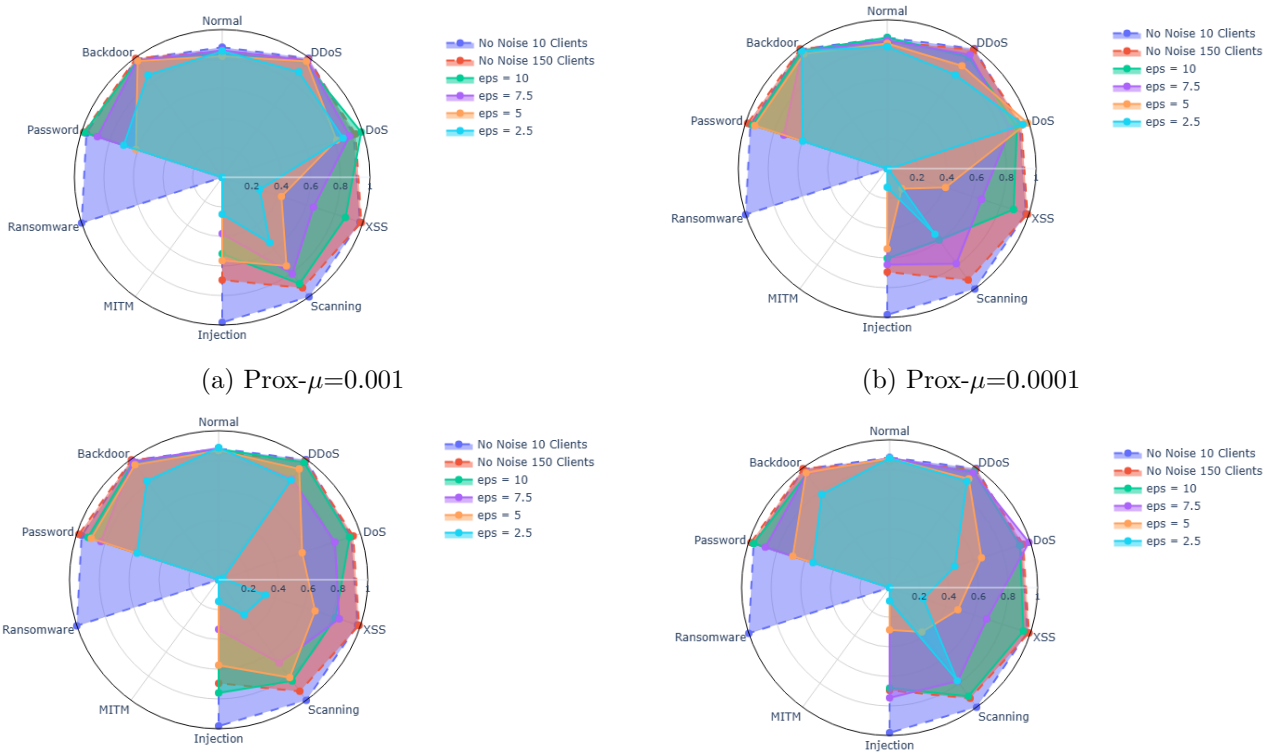


(g) ϵ=2.5, SGD



(h) ϵ=2.5, Adam

Figure 9: Training Loss across ϵ



(a) Prox-μ=0.001

(b) Prox-μ=0.0001

(c) Prox-μ=0.00001

(d) Prox-μ=0

Figure 10: Accuracy Per Class

epochs doesn't have an effect on model performance beyond baseline conditions. This aligns with the findings in [8]. Changing the other parameters causes the attack results to diverge: multi-round attacks perform better than DLG and, DLG with cosine similarity performs the best.

All measures of PCC are close to zero – there is no correlation between recovered and original inputs. None of the attacks are successful even in baseline conditions.

Tables 6 and 7 show the results of experiment 8. The experiment was run with 150 clients to stabilise differentially private learning and 10 of these clients were sampled for attacks. Baseline learning parameters were used (row 1 in Figure 5). The attacks seem to perform worse for lower levels of noise; the distance between original and recovered inputs peaks at $\epsilon = 7.5$. This is likely not indicative of the true privacy of the system. Instead, applying noise to gradients in the early rounds of training artificially creates much larger gradients; the attack, in an attempt to minimise the distance between dummy gradients and the large client gradients, increases the magnitude of the inputs. This effect is more exaggerated for medium to low noise because the gradients will be large but not as large, so exploding the inputs doesn't quickly find a minima and the attack continues to increase the magnitude of the inputs. This would also explain why, when using cosine similarity as the distance metric, the attack remains at baseline conditions: still not learning beyond dummy inputs and why SGD resulted in worse recovery than Adam (it is more sensitive to noise).

5 Reflection

5.1 Extent to which the project aims have been achieved

Aim 1 - Competitive Baseline Model: My best models – using both SGD and Adam as optimizers – achieved an accuracy $> 95\%$ and an f1-score $> 75\%$ in 35 rounds. I showed the baseline model using Adam scales well across clients and achieves a macro-recall $> 80\%$ (this doesn't scale across clients, but

Table 5: Recoverable Information Across Local Training Parameters and Attacks

Parameters				**Attack Performance**								
				DLG			**CosSim**			**Multi**		
Batch Size	Local Epochs	Local Batches	Window Size	Avg. MSE	Min. MSE	Avg. PCC	Avg. MSE	Min. MSE	Avg. PCC	Avg. MSE	Min. MSE	Avg. PCC
1	1	1	20	**1.5140**	**1.12530**	**-0.005249**	**1.5352**	**1.1223**	**-0.0095**	**1.5138**	**1.1223**	**-0.0052**
1	1	8	20	2.8841	1.4087	-0.0034	1.7015	1.1626	-0.0042	2.8358	1.3830	-0.0014
1	1	16	20	4.7345	2.0263	-0.00103	2.8629	1.1914	-0.0019	4.0570	1.6609	-0.0045
1	1	32	20	9.6894	1.8108	-0.0026	2.7591	1.2759	-0.0001	4.3727	1.7274	0.0005
1	1	64	20	13.0359	2.0882	-0.0018	2.1554	1.2123	-0.0007	5.6647	2.1255	-0.0019
8	1	1	20	**3.4340**	**1.4123**	**0.0003**	**3.1914**	**1.2194**	**-0.0032**	**3.3352**	**1.3002**	**-0.0032**
16	1	1	20	3.3637	1.6163	-0.0089	2.8908	1.2247	-0.0015	3.2942,	1.4953	-0.0060
32	1	1	20	3.8435	1.6461	0.0061	3.0317	1.2944	-0.0016	3.718	1.7719	0.0042
64	1	1	20	3.2992	2.1103	0.0053	2.2658	1.2090	0.0005	3.1281	2.2473	0.0056
1	3	1	20	**1.6421**	**1.1439**	**-0.0003**	**1.7780**	**1.1442**	**0.0067**	**1.6421**	**1.1444**	**-0.0004**
1	5	1	20	1.3512	1.1336	-0.0021	1.3228	1.1336	0.0072	1.3511	1.1336	-0.0022
1	7	1	20	1.4945	1.1321	0.0058	1.4955	1.1403	0.0060	1.4942	1.1322	0.0059
1	10	1	20	3.8849	1.1309	-0.0089	6.5212	1.1478	-0.0062	3.8848	1.1309	-0.0089
1	1	1	30	**1.3431**	**1.1192**	**0.0011**	**1.6134**	**1.1037**	**-0.0047**	**1.5892**	**1.1236**	**-0.0072**
1	1	1	40	1.4685	1.1807	-0.0099	861.960	1.1244	-0.0033	2.5451	1.1429	-0.0032
1	1	1	50	3.5443	1.1879	0.0060	3.4000	1.1602	0.0020	3.4491	1.1937	0.0031

Table 6: Recoverable Information Across Noise and Inversion Attacks (Adam)

Parameters	**Attack Performance**								
	DLG			**CosSim**			**Multi**		
ϵ	Avg. MSE	Min. MSE	Avg. PCC	Avg. MSE	Min. MSE	Avg. PCC	Avg. MSE	Min. MSE	Avg. PCC
2.5	**1327.9**	**3.5068**	**-0.004850**	**1.31855**	**1.1490**	**-0.010640**	**1552.6**	**267.4928**	**-0.011664**
5.0	14899.5	2533.750	-0.006025	1.26170	1.1315	0.008553	19492.5	1292.4500	0.020043
7.5	30984.0	4719.0500	-0.008798	1.52805	1.1218	0.002423	19905.0	10569.0000	-0.003344
10.0	20343.5	1195.0554	0.008668	1.20490	1.1321	0.005351	19224.0	6256.9500	0.001736

Table 7: Recoverable Information Across Noise and Inversion Attacks (SGD)

Parameters	**Attack Performance**								
	DLG			**CosSim**			**Multi**		
ϵ	Avg. MSE	Min. MSE	Avg. PCC	Avg. MSE	Min. MSE	Avg. PCC	Avg. MSE	Min. MSE	Avg. PCC
2.5	**886.475**	**5.22305**	**0.009068**	**1.22400**	**1.14655**	**0.002913**	**596.915**	**44.5566**	**-0.005820**
5.0	14310.000	4415.75000	-0.007202	1.28555	1.14495	-0.002470	11004.000	2361.1500	-0.008735
7.5	34984.500	10218.55000	-0.014510	1.25215	1.12610	0.005027	28191.500	4679.3000	-0.011004
10.0	32875.500	4621.30500	-0.002575	1.20050	1.12840	0.000032	24836.500	9770.3500	-0.009963

that is due to partitioning logic). This demonstrates the viability of my model and FL-Framework in federated network intrusion detection tasks. This aim and all the objectives were successfully achieved.

Aim 2 - Empirical Study of the privacy-utility trade-off in Federated Intrusion Detection:

This aim was partially achieved. I have clearly demonstrated how local differential privacy affects the viability of the model by measuring the stability of learning as the number of clients varies and measuring the bias of the model across noise. However, I am limited in the number of clients I can simulate; beyond 150 clients the local datasets are too sparse and the partitioner fails. This means I can't show exactly at how many clients each level of noise starts to learn better. The implemented attacks do not provide an accurate measure privacy and struggle to recover inputs in baseline conditions.

Aim 3 - Privacy Preserving Framework: Using Adam and the FedProx aggregation scheme I have created an FL-based IDS that demonstrates strong accuracy across noise and mitigates bias for a high number of clients. However, the framework only achieves $> 80\%$ accuracy and a macro-recall $> 70\%$ for $\epsilon = 10$ and a high number of clients. Due to the complexity of heterogeneous network traffic data and the model architecture, inversion attacks fail to recover inputs even in baseline conditions. This aim was partially achieved.

5.2 Challenges

Network Traffic Data: The size of and complexity of the dataset and distribution of classes have been a significant challenge of this project. While this was somewhat mitigated by the pre-processing pipeline, there is still significant bias. Partitioning the ToN_IoT dataset into fully realistic network traffic time series' was not possible due to sequential class collection.

Inversion Attacks: A drawback of using inversion attacks to quantify privacy is its dependence on an adversary's capabilities. After tuning attacks and experimenting with different optimizers I couldn't successfully recover inputs in baseline conditions. I show how the recoverable information changes across different privacy mechanisms and FL settings however this is undermined by practicalities such as computational limitations in simulating attacks on many clients with large training data and a weak baseline.

6 Conclusions

I have shown how local differential privacy negatively effects the final global accuracy of the model and the bias towards simpler and prominent attacks, and that differential privacy is unsuitable when training among fewer clients. I have also demonstrated that these effects can be mitigated by using advanced optimization methods and adding regularization terms to limit divergence in client updates.

By using inversion attacks to quantify privacy, I show that varying local batches and batch sizes has a privacy-preserving effect. The DLG attack on models trained over 64 batches demonstrated an MSE between original and recovered inputs an order of magnitude greater than the baseline. However, the empirical approach to quantifying privacy could potentially yield overly optimistic results by relying on existing attacks instead of bounds on recoverable information.

Nonetheless, we show the definition of DP does not provide a complete measure of privacy in FL-based IDSs. There is already a measure of privacy inherent in the complexities of model inputs and the model architecture, especially so in heterogeneous network traffic data. Future work aims to better quantify this effect. By focusing on realistic heterogeneous datasets and using a complex CNN architecture for classification, the baseline for measuring how recoverable information is affected as complexity scales is already too high. Next steps will start with much simpler models and idealised network traffic data and grow the complexity to provide a better understanding of privacy in FL-based IDSs.

This is not to say that FL-IDS's are inherently private due to their complexities, but that there is a need for improved measures of privacy in collaborative learning so we can better understand when differential privacy is needed and how much noise is needed. Work is already being done to address this in split inference learning [10], I am not aware of any attempts made in federated learning.

References

[1] B. Krebs "Hacked Cameras, DVRs Powered Today's Massive Internet Outage" KrebsonSecurity, Oct. 21, 2016. [Online]. Available: https://krebsonsecurity.com/2016/10/hacked-cameras-dvrs-powered-todays-massive-internet-outage/

[2] R.C. Geyer, T. Klein, M. Nabi, "Differentially Private Federated Learning: A Client Level Perspective," 2017, *arXiv:1712.07557*. [Online]. Available: https://arxiv.org/abs/1712.07557 .

[3] P. Ruzafa-Alcázar et al., "Intrusion Detection Based on Privacy-Preserving Federated Learning for the Industrial IoT," in *IEEE Transactions on Industrial Informatics,* vol. 19, no. 2, pp. 1145-1154, Feb. 2023, doi: 10.1109/TII.2021.3126728.

[4] J. Chen, Y. Zhao, Q. Li, X. Feng and K. Xu, "FedDef: Defense Against Gradient Leakage in Federated Learning-Based Network Intrusion Detection Systems," in *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 4561-4576, 2023, doi: 10.1109/TIFS.2023.3297369.

[5] W. Yao, H. Zhao and H. Shi, "Privacy-Preserving Collaborative Intrusion Detection in Edge of Internet of Things: A Robust and Efficient Deep Generative Learning Approach," in *IEEE Internet of Things Journal*, vol. 11, no. 9, pp. 15704-15722, 1 May1, 2024, doi: 10.1109/JIOT.2023.3348117.

[6] I. A. Soomro et al., "ROCHE: A Robust and End-to-End Privacy-Preserving Federated Learning Framework for Intrusion Detection in Industrial Internet of Things," in *IEEE Internet of Things Journal*, doi: 10.1109/JIOT.2025.3612944.

[7] B. Li, Y. Wu, J. Song, R. Lu, T. Li and L. Zhao, "DeepFed: Federated Deep Learning for Intrusion Detection in Industrial Cyber–Physical Systems," in *IEEE Transactions on Industrial Informatics*, vol. 17, no. 8, pp. 5615-5624, Aug. 2021, doi: 10.1109/TII.2020.3023430.

[8] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients – how easy is it to break privacy in federated learning?," in *Proc. 34th Int. Conf. Neural Information Processing Systems (NeurIPS)*, Vancouver, Canada, 2020, pp. 1421–1431.

[9] E. Bagdasaryan, and V. Shmatikov, "Differential Privacy Has Disparate Impact on Model Accuracy," 2019, *arXiv:1905.12101*. [Online]. Available: https://doi.org/10.48550/arXiv.1905.12101 .

[10] R. Deng, Z. Lu, and Q. Duan, "Quantifying Privacy Leakage in Split Inference via Fisher-Approximated Shannon Information Analysis" 2025, *arXiv:2504.10016*. [Online]. Available: https://doi.org/10.48550/arXiv.2504.10016 .

[11] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. NeurIPS*, 2019, pp. 1-11.

[12] J. Geng, Y. Mou, Q. Li, F. Li, O. Beyan, S. Decker, C. Rong "Imporved Gradient Inversion Attacks and Defenses in Federated Learning." in *IEEE Transactions on Big Data* , vol. 10, no. 6, pp. 839-850, Nov. 2024, doi: 10.1109/TBDATA.2023.3239116.

[13] M. Veale, R. Binns, and L. Edwards "Algorithms that remember: model inversion attacks and data protection law" *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 376(2133): 20180083, 2018. `https://doi.org/10.1098/rsta.2018.0083`

[14] M. Abadi, A. Chu, I. J. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2016, pp. 308–318.

[15] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh, A. Dehghantanha and G. Srivastava, "Federated-Learning-Based Anomaly Detection for IoT Security Attacks," in *IEEE Internet of Things Journal,* vol. 9, no. 4, pp. 2545-2554, 15 Feb.15, 2022, doi: 10.1109/JIOT.2021.3077803.

[16] C. Sanders, and J. Smith "Applied Network Security Monitoring", Syngress, 2014, ch. 4, pp. 75-97.

[17] T. M. Booij, I. Chiscop, E. Meeuwissen, N. Moustafa and F. T. H. d. Hartog, "ToN_IoT: The Role of Heterogeneity and the Need for Standardization of Features and Attack Types in IoT Network Intrusion Data Sets," in *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 485-496, 1 Jan.1, 2022, doi: 10.1109/JIOT.2021.3085194.

[18] R. Malouf, "A comparison of algorithms for maximum entropy parameter estimation" in *Proc. 6th Conf. Natural Language Learning (CoNLL-02)*, 2002, pp.1-7, doi: 10.3115/1118853.1118871.

[19] D. C. Liu, and J. Nocedal, "On the limited memory BFGS method for large scale optimization," *Math. Program.,* vol. 45, no. 1, pp. 503-528, 1989.

[20] Flower Labs, "Flower Architecture," *Flower Documentation.* [Online]. Available: `https://flower.ai/docs/framework/explanation-flower-architecture.html`.

[21] J. Van Der Donckt, J. Van Der Donckt, E. Deprost, and S. Van Hoecke, "tsflex: Flexible time series processing & feature extraction," *SoftwareX*, vol. 17, Art. no. 100971, 2022, doi: 10.1016/j.softx.2021.100971.

[22] T. Li, A. k. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated Optimization in Heterogeneous Networks," in *Proc. Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 429-450.

[23] Y. Sharon, D. Berend, Y. Liu, A. Shabtai and Y. Elovici, "TANTRA: Timing-Based Adversarial Network Traffic Reshaping Attack," in *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 3225-3237, 2022, doi: 10.1109/TIFS.2022.3201377.

[24] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. 3rd Theory of Cryptography Conference (TCC)*, New York, NY, 2006, pp. 265–284, doi: 10.1007/11681878_14.

[25] C. Dwork, K. Kenthapadi, F. McSherry, I. Mironov, M. Naor "Our Data, Ourselves: Privacy Via Distributed Noise Generation," in *Proc. Advances in Cryptology* - EUROCRYPT 2006, vol. 4004, pp. 485-503.

[26] S. Kiranyaz, O. Avci, O. Abdeljaber, T. Ince, M. Gabbouj, D J. Inman, "1D convolutional neural networks and applications: A survey," in *Mechanical Systems and Signal Processing*, 2021, vol. 151, pp. 107398.

[27] A. Naeem, J. Ahmad, M. A. Khan, A. A. Khattak, and M. S. Khan "Efficient IoT Intrusion Detection with an Improved Attention-Based CNN-BiLSTM Architecture," *arXiv preprint arXiv:2503.19339,* 2026.

[28] C. C. Aggarwal, "Advanced Optimization Solutions," in *Linear Algebra and Optimization for Machine Learning,*, Cham, Switzerland: Springer International Publishing, 2020, pp. 205-253.

[29] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. Buarque de Gusmão, and N. D. Lane, "Flower: A Friendly Federated Learning Research Framework," 2020, *arXiv:2007.14390.* [Online]. Available:`https://doi.org/10.48550/arXiv.2007.14390`

[30] R. Aburukba, A. R. Al-Ali, N. Kandil and D. AbuDamis, "Configurable ZigBee-based control system for people with multiple disabilities in smart homes," *2016 International Conference on Industrial Informatics and Computer Systems (CIICS)* , Sharjah, United Arab Emirates, 2016, pp. 1-5, doi: 10.1109/ICCSII.2016.7462435.

[31] A. Gatouillat, Y. Badr, B. Massot and E. Sejdić, "Internet of Medical Things: A Review of Recent Contributions Dealing With Cyber-Physical Systems in Medicine," in *IEEE Internet of Things Journal,* vol. 5, no. 5, pp. 3810-3822, Oct. 2018, doi: 10.1109/JIOT.2018.2849014.

[32] T. T. Dandala, V. Krishnamurthy and R. Alwan, "Internet of Vehicles (IoV) for traffic management," *2017 International Conference on Computer, Communication and Signal Processing (ICCCSP),* Chennai, India, 2017, pp. 1-4, doi: 10.1109/ICCCSP.2017.7944096.

[33] A. Humayed, J. Lin, F. Li and B. Luo, "Cyber-Physical Systems Security—A Survey," in *IEEE Internet of Things Journal,* vol. 4, no. 6, pp. 1802-1831, Dec. 2017, doi: 10.1109/JIOT.2017.2703172.

[34] H. Sun, M. Li "Precision Agriculture in China: Sensing Technology and Application," in *Precision Agriculture Technology for Crop Farming*, CRC Press, 2016, pp 232-275.

[35] B. Lampe and W. Meng, "Intrusion Detection in the Automotive Domain: A Comprehensive Review," in *IEEE Communications Surveys & Tutorials,* vol. 25, no. 4, pp. 2356-2426, Fourthquarter 2023, doi: 10.1109/COMST.2023.3309864.

[36] R. Langner, "Stuxnet: Dissecting a Cyberwarfare Weapon," in *IEEE Security & Privacy,* vol. 9, no. 3, pp. 49-51, May-June 2011, doi: 10.1109/MSP.2011.67.

List of Prompts:

"How is this conclusion ..."

`https://chatgpt.com/share/69f19791-e874-83eb-8681-975a0fe7c855`

"What is the best way to cite tsflex ..."

`https://chatgpt.com/share/69f19865-1a50-83eb-a6f5-9f637bc9cc11`

"How can I add a footnote to the figure caption, ..."

`https://chatgpt.com/share/69f198cc-0614-83eb-8ab1-3c4d9b79d6e3`

"Can you improve this table for me: ..."

`https://chatgpt.com/share/69f198ff-fc80-83eb-991e-72f92c27f7a7`

"... what part of the docs would be most helpful?"

`https://chatgpt.com/share/69f19a0c-d358-83eb-907f-f9e2db4b1669`

Appendix A

Table 8: Hyperparameter Search Results with Adam optimiser - Full Results

Hyperparameters				**Performance**	
LR	Local Epochs	Batch Size	Window Size	Global Accuracy	F1-Score
0.001	10	32	20	**0.95312**	**0.87348**
0.001	3	64	30	0.94996	0.76361
0.001	10	16	40	0.94995	0.75168
0.001	10	16	50	0.94778	0.75108
0.0001	5	32	20	0.94688	0.75204
0.0001	3	32	50	0.94125	0.73208
0.0001	5	32	50	0.93995	0.72103
0.001	10	32	40	0.93639	0.76262
0.001	5	32	40	0.93326	0.74489
0.0001	1	16	20	0.93021	0.70381
0.0001	3	64	30	0.92651	0.70893
0.01	1	64	40	0.92075	0.7078
0.01	5	16	30	0.91634	0.69425
0.01	5	64	30	0.90305	0.71168
0.01	5	32	30	0.89289	0.68043
0.01	1	16	40	0.89155	0.64352
0.01	1	32	40	0.89155	0.66097
0.001	10	64	30	0.88819	0.6795
0.01	5	64	50	0.84595	0.66639
0.0001	1	64	30	0.83581	0.43323
0.0001	1	32	50	0.82507	0.44009
0.0001	1	32	40	0.82377	0.43386
0.03	3	32	40	0.76434	0.48157
0.1	1	64	30	0.74824	0.32586
0.1	1	64	40	0.62357	0.23403
0.03	3	32	50	0.57963	0.3644
0.1	5	16	30	0.41673	0.05883
0.1	10	16	50	0.40992	0.058148
0.1	3	64	40	0.40667	0.1378
0.1	3	64	30	0.39406	0.20242

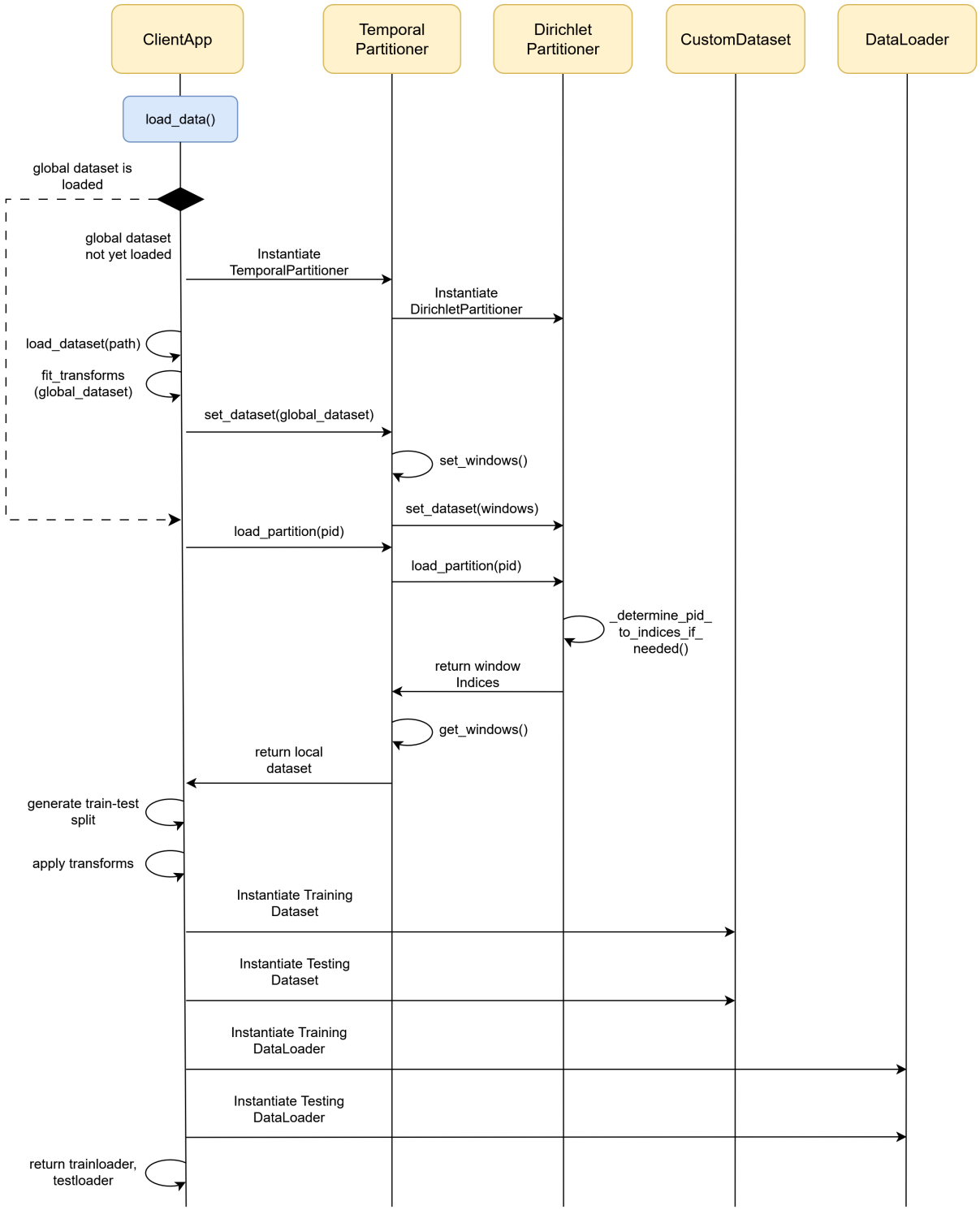


Figure 11: Data Loading Activity Diagram

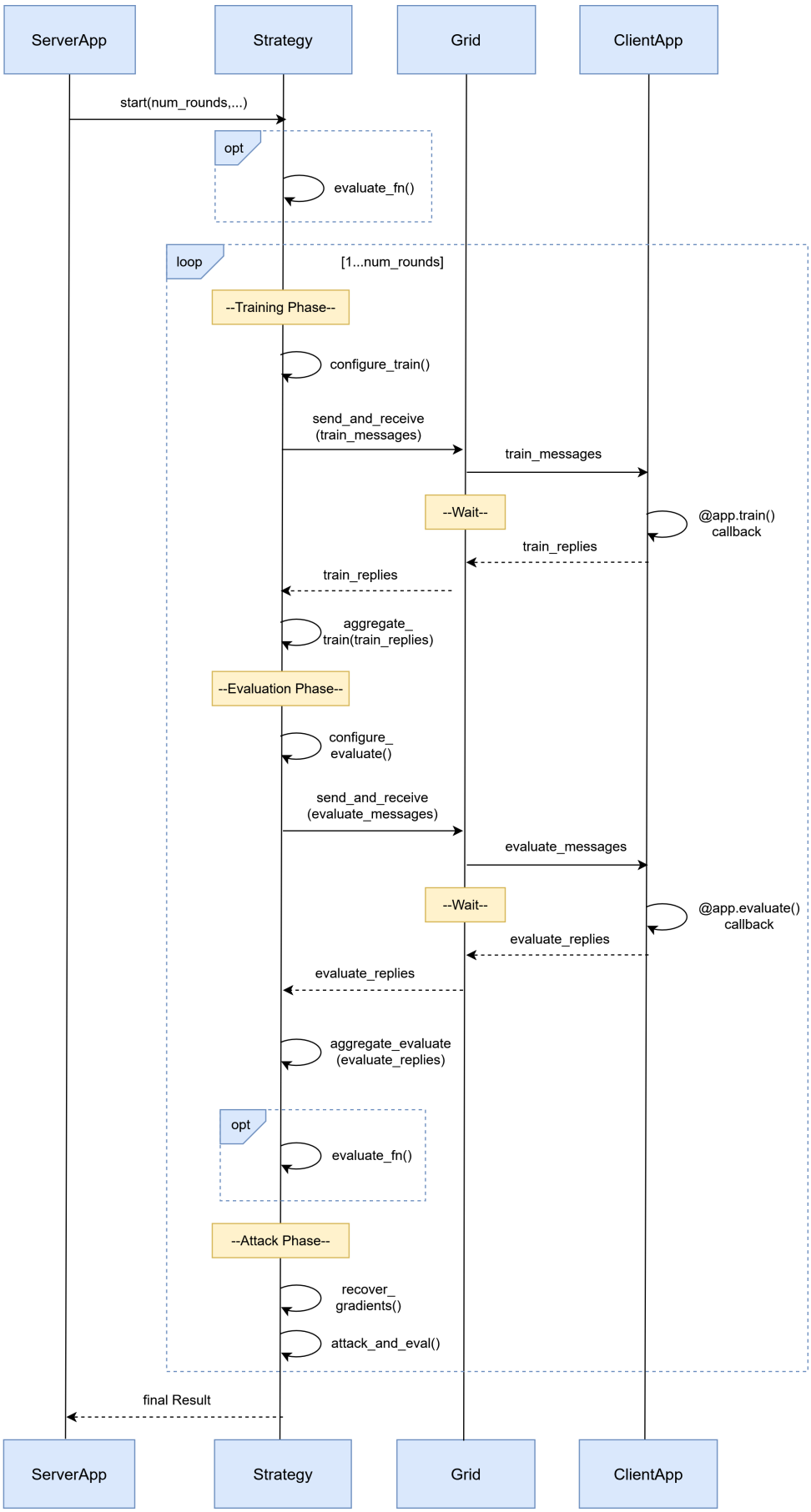


Figure 12: Start Function with Attack

Table 9: Hyperparameter Search Results with SGD optimiser - Full Results

Hyperparameters				**Performance**	
LR	Local Epochs	Batch Size	Window Size	Global Accuracy	F1-Score
0.01	10	64	20	**0.96302**	**0.76281**
0.01	3	16	50	0.96084	0.75522
0.1	5	32	30	0.95856	0.84063
0.01	10	16	50	0.95300	0.76461
0.03	5	16	40	0.94891	0.76503
0.01	5	16	20	0.94740	0.75900
0.1	1	32	20	0.94531	0.76227
0.1	3	32	30	0.93980	0.7485
0.01	5	32	50	0.93603	0.73763
0.1	5	64	40	0.93535	0.72533
0.03	1	32	30	0.93432	0.72602
0.001	10	16	20	0.93281	0.72048
0.01	5	32	40	0.92701	0.71909
0.01	3	64	30	0.92338	0.67745
0.03	1	64	30	0.92338	0.71021
0.01	3	64	50	0.92037	0.68394
0.01	3	16	40	0.91554	0.7008
0.1	10	64	40	0.91345	0.71999
0.001	10	64	20	0.88750	0.59028
0.001	10	64	30	0.82799	0.46995
0.001	10	64	40	0.81543	0.41872
0.001	3	32	40	0.81335	0.42604
0.001	5	64	30	0.81001	0.42173
0.0001	5	16	20	0.63438	0.25238
0.0001	1	16	50	0.62402	0.29028
0.0001	10	16	40	0.5829	0.20566
0.0001	3	16	50	0.42689	0.061955
0.0001	1	32	20	0.42135	0.070572
0.001	1	64	40	0.41919	0.060681
0.0001	3	32	40	0.40146	0.058956

Table 10: Differential Privacy Parameter Search Results with Adam optimiser - Full Results

Hyperparameters			**Performance**		
Sensitivity	Clipping Norm	δ	Global Accuracy	F1-Score	Final Loss
0.5	2	1e-05	**0.91042**	**0.71557**	**1.295**
0.5	1	1e-05	0.86615	0.61034	1.3681
1	2	1e-06	0.85000	0.58718	16.096
1	2	1e-05	0.78646	0.53616	14.397
1	1	1e-05	0.78333	0.48922	29.377
0.5	0.5	1e-06	0.76771	0.49869	2.6172
2	10	1e-06	0.7625	0.47013	1288.1
2	2	1e-06	0.7625	0.40116	1749.6
2	5	1e-05	0.76146	0.50554	1492.3
2	5	1e-06	0.73958	0.40550	1257.2
4	5	1e-06	0.73594	0.44598	68072.0
2	10	1e-05	0.71771	0.42848	645.25
1	10	1e-06	0.71562	0.41302	34.167
4	2	1e-06	0.6849	0.38026	106380.0
4	2	1e-05	0.68125	0.33843	89667.0
4	5	1e-05	0.6375	0.37948	40034.0
2	0.5	1e-05	0.6276	0.31922	4242.4
4	10	1e-05	**0.58802**	**0.31611**	**65442.0**
2	0.5	1e-06	0.5875	0.34124	4295.3
4	1	1e-05	0.46563	0.22964	94696.0

Table 11: Attack Hyperparameter Search Results with LBFGS optimizer

Hyperparameters			**Performance**		
Rounds	Max Iters	History Size	Avg. MSE	Min. MSE	Avg. PCC
50	20	10	**1.1822**	**1.1256**	**0.0062**
25	10	50	1.2632	1.1141	-0.0081
25	5	100	1.2691	1.1392	-0.0174
25	20	50	1.2787	1.1481	-0.0199
25	20	10	1.2928	1.1214	-0.0013
50	20	50	1.2945	1.1189	-0.0049
50	20	100	1.2990	1.1511	-0.0125
50	10	100	1.3668	1.1430	0.0045
10	5	50	1.3728	1.1237	0.0101
10	10	50	1.4094	1.1417	0.0086
10	5	10	1.4525	1.1228	0.0090
25	10	100	1.4897	1.1265	0.0007
50	10	10	1.5219	1.1543	-0.0114
10	5	100	1.5419	1.1286	0.0252
10	20	10	1.6922	1.1429	0.0125
25	5	10	1.9842	1.1130	-0.0056
10	20	50	4.2944	1.1585	-0.0005
50	5	10	6.3314	1.1308	0.0009
50	20	10	7.3489	1.0959	0.0033
50	5	100	N/A	1.1187	N/A